The Task Completion Problem and its Application to Crash-Resilient Computation

Orr Fischer
Bar-Ilan University
orr.fischer@biu.ac.il

Ran Gelles
Bar-Ilan University
ran.gelles@biu.ac.il

arXiv:2605.17961v1 [cs.DC] 18 May 2026

Abstract

We study the *Task Completion* problem, in which M abstract tasks must be completed by a network of n crash-prone nodes, where up to αn nodes may crash for some constant $\alpha < 1$. Our main result is a deterministic congested-clique algorithm that completes all M tasks in $O(\lceil M/n \rceil \log n)$ rounds. This round complexity is optimal up to $\log \log n$ terms.

The key technical ingredient underlying our algorithm is a novel combinatorial structure, which we call a *load balancing covering family*. In essence, this covering family induces, for each task, a subset of nodes responsible for attempting to complete it. The properties of the load balancing covering family guarantee that, regardless of which tasks remain incomplete and which nodes crash, (i) no node is overloaded with incomplete tasks, and (ii) no task is left with too few potential assigned nodes. This yields a balanced per-node workload and prevents non-crashed nodes from being concentrated on a small subset of tasks, thereby ensuring sufficient progress in completing the remaining tasks.

As an application of our task completion method, we give a deterministic algorithm for simulating any T-round congested-clique algorithm in the presence of up to αn crash faults in $O(T^2 \log n + T \log^2 n)$ rounds. This improves upon a recent result by Censor-Hillel et al. (DISC 2025), which requires $T^2 \cdot 2^{O(\sqrt{\log n} \log \log n)}$ rounds.

1 Introduction

One of the main advantages of distributed computation is the ability to parallelize work across multiple concurrent devices that operate simultaneously and independently, resulting in a significantly reduced overall completion time. In particular, if n parallel tasks can be distributed among a network of n computing devices (nodes), and each task requires constant time, then completing all tasks in parallel reduces the total time to $O(1)$. This idealized scenario is often disrupted by *faults*. Some nodes may malfunction and stop operating during the computation (i.e., nodes may *crash*). Another potential challenge is *congestion*: when communication links have bounded bandwidth, the remaining nodes may become bottlenecks for both incoming and outgoing communication. In this work, we study crash-prone *congested cliques* [40], where up to an $\alpha < 1$ fraction of the n fully connected nodes may crash, and seek efficient (ideally, optimal) algorithms for completing all tasks under the $O(\log n)$ per-message bandwidth restriction.

At first glance, and ignoring congestion for the moment, crashes might appear to have only a limited impact on the overall computation time, even when the fraction of failing nodes is large. If the αn devices that may fail were known in advance, the remaining $(1-\alpha)n$ nodes could redistribute the workload among themselves and complete the n tasks in $O(1/(1-\alpha))$ rounds. While this quantity grows as $\alpha \to 1$, it remains constant.

Unfortunately, this intuition is misleading, as it relies on advance knowledge of which αn nodes will crash, allowing the remaining nodes to take over their work. In practice, the set of crashed nodes is not known ahead of time and can occur adaptively throughout the computation. Moreover, nodes may fail in the middle of a round, leading to inconsistent views among the remaining nodes. As a result, at any given round, there may be no consensus on which nodes have crashed.

When up to αn nodes may crash arbitrarily, the remaining nodes must coordinate in order to detect these failures and redistribute the yet-incomplete tasks. This coordination itself incurs a nontrivial cost: it requires $\Omega(\log n)$ rounds, even if all nodes agree in every round on which nodes have crashed so far and, consequently, on which tasks remain incomplete; see Section 3.2 of [26].

Message-Passing Task Completion and Prior Work. The *Task Completion Problem* (also known as *Do-All*) considers the setting in which M *tasks* must be completed by a fully-connected network of n nodes despite crash failures.[1] Tasks are *abstract* entities that can represent actions within the model (e.g., sending a message or updating a local variable), but may also be used to represent activities outside the model, such as accessing a shared resource, writing to an external device, etc. A task is typically specified by a predicate whose satisfaction indicates that the task is complete, together with pseudocode describing how the completion of the task can be achieved. Examples of such tasks appear in Task-Completion Instances 1–3 in Section 5. Tasks need to be *independent*: any node can complete any task, with no prescribed order or dependencies. Moreover, up to n tasks may be completed simultaneously (e.g., when all nodes attempt to complete different tasks in the same round). Tasks need also to be *idempotent*: if a task is completed multiple times or concurrently by multiple nodes, the outcome is the same and this event is counted as a single correct completion of the task. We assume completion of a single task takes $R \geq 1$ rounds and require tasks to be *executor-safe*: if a node attempts to complete a task, and that node does not crash during the R rounds of its execution, then the completion of the task is guaranteed regardless

[1] To simplify the presentation, in this section we set $M = n$. We stress that all results also apply to the general case.

of failures of other nodes. Note that the task may involve actions of other nodes, however, the task is executor-safe if their failure does not prevent the correct completion of the task.

The task completion problem was introduced for synchronous message-passing systems by Dwork, Halpern, and Waarts [21]. They designed three different algorithms that tradeoff time and message complexity while aiming to optimize the *work*, defined as the total number of attempts to complete tasks (each node can perform one unit of work per round or remain idle). Assuming $R = 1$, their first algorithm runs in $O(n)$ rounds, while the second requires exponentially many rounds. The round complexity of their third algorithm depends on the number of faults that actually occur: it takes $O(n^2)$ rounds in the worst case, but only $O(1)$ rounds in the fault-free scenario. Later works [19, 11, 18, 12] typically do not analyze the total round complexity, but instead state only the work, which implies loose bounds on the round complexity. These works take at least $\Omega(\log^2 n)$ rounds to complete all tasks.[2] The work of Georgiou, Russell, and Shvartsman [25] suggests that $O(\log n)$ rounds may suffice if the nodes are equipped with an oracle for incomplete tasks, that is, if the nodes are always in consensus regarding which tasks remain incomplete. However, finding an algorithm with $O(\log n)$ round complexity without access to such an oracle remains an important open problem, and to the best of our knowledge, no deterministic $O(\log n)$-round solution in the message-passing model is currently known.

We answer the above open question affirmatively by presenting a deterministic message-passing algorithm for the task completion problem that completes in $O(\log n)$ rounds when $R = 1$ in the presence of up to αn crashes without any other assumptions. Furthermore, in our algorithm, nodes communicate using messages of size $O(\log n)$ bits; that is, the algorithm fits within the *congested clique* model [40]. We can now, somewhat informally, state our main result as follows.

Theorem 1.1. *For any $\alpha < 1$, if completing a single task takes $R \geq 1$ rounds, the n-task completion problem can be solved by a deterministic congested clique algorithm in $O(R \log n)$ rounds in the presence of up to αn crash-faults.*

We note that this result is tight up to $\log \log n$ factors. Indeed, a lower bound of $\Omega(\log n / \log \log n)$ rounds for the n-task completion problem with $R = 1$ was established by Georgiou, Russell, and Shvartsman [25, 26], even in the stronger setting where nodes share a global view of the incomplete tasks via an oracle. In Section B, we revisit this proof in a slightly simplified form for the sake of completeness.

While the task completion problem is a fundamental distributed problem in its own right, it is also closely connected to resilient simulation of algorithms, particularly in the PRAM model (see, e.g., [26, 32] and the related work section below). We leverage our congested-clique task completion algorithm to obtain a method for simulating any congested-clique algorithm in crash-prone networks, while maintaining low bandwidth usage to avoid congestion. This application is summarized in the following theorem.

Theorem 1.2. *For any $\alpha < 1$, any congested-clique algorithm that runs in T rounds can be simulated in the presence of up to αn crash-faults in $O(T^2 \log n + T \log^2 n)$ rounds.*

The above theorem improves upon a recent result by Censor-Hillel, Fischer, Gelles, and Soto [6], which simulates any T-round algorithm in $T^2 \cdot 2^{O(\sqrt{\log n} \log \log n)}$ rounds. In another recent work,

[2] Note that many of these works tolerate a number of failures $f < n$, which trivially implies $\Omega(n)$ rounds for the case where only a single node remains to complete all tasks. For a fair comparison, we consider bounds that explicitly depend on the number of failures f and substitute $f = \alpha n$.

Censor-Hillel and Soto [9] show that in certain special cases, structural properties of the simulated algorithm allow for further optimization, yielding simulations with only polylogarithmic (in n) overhead. In the same spirit, our simulation technique also achieves improved performance in specific cases. In particular, when the state maintained by each node has size at most $O(n \log n)$ bits, our simulation runs in $O(T \log^2 n)$ rounds, incurring only polylogarithmic overhead.

1.1 Our Techniques

A load balancing covering family. Our task completion algorithm relies on a new combinatorial structure that we call a *load balancing covering family*. This covering family consists of sets $A_1, \ldots, A_m$, where each A_i is a subset of $\{1, \ldots, n\}$. The covering family is parameterized by three values: integers k and B, and a fraction $\epsilon \in (0, 1)$, and it satisfies two important and useful properties (see Definition 1 for a formal statement). (i) Each set A_i is neither too large nor too small; specifically, for all i, $\frac{nB}{2k} \leq |A_i| \leq \frac{2nB}{k}$. (ii) Any k sets $\{A_{i_1}, \ldots, A_{i_k}\}$ are load-balanced with respect to a set $J \subseteq \{1, \ldots, n\}$ of size at least $(1-\epsilon)n$. By "load-balanced" we mean that each element $j \in J$ appears in approximately B of the subsets $\{A_{i_1}, \ldots, A_{i_k}\}$. Formally, for all $j \in J$,

$$(1-\epsilon)B \leq |\{\ell \in \{1, \ldots, k\} \mid j \in A_{i_\ell}\}| \leq (1+\epsilon)B.$$

A load balancing covering family–based task completion. We use the load balancing covering family as follows. Assume there are n nodes and m tasks in total, out of which $k \leq m$ tasks are incomplete. For the time being, suppose that the nodes have full knowledge of which tasks still need to be completed; we later show how to remove this assumption. The nodes construct a (k, B, ϵ)-load balancing covering family with $0 < \epsilon < 1 - \alpha$ and $B = O(1)$. The existence of such a family is established in Section 3 using the probabilistic method.

Given this construction, task completion proceeds as follows. For $2B \cdot R$ rounds, node j attempts to complete all incomplete tasks $i \in \{1, \ldots, m\}$ such that $j \in A_i$. That is, for each task i, the set A_i specifies the nodes assigned to complete task i. The second property of the load balancing covering family guarantees that all but ϵn nodes are assigned at most $(1+\epsilon)B$ incomplete tasks and can therefore complete all of them within these $2B \cdot R$ rounds, unless they crash (recall, completing a single task takes $R \geq 1$ rounds). Moreover, since $\epsilon < 1 - \alpha$, even if αn nodes have already crashed, the set J still contains $(1 - \epsilon - \alpha)n = \Omega(n)$ nodes that can make progress during this segment. The first property of the load balancing covering family ensures that nodes are well distributed across tasks: no task is assigned too many nodes, and the assignments are instead evenly spread. Hence, the $\Omega(n)$ non-crashed nodes in J are not all assigned to the same task(s) and can make substantial progress during this step. In particular, we prove that at least ϵk of the incomplete tasks are completed.

After this step, we recurse on the remaining incomplete tasks. Each step reduces the number of incomplete tasks by a factor of ϵ, and therefore after $O(\log_{1/\epsilon} n)$ steps, all tasks are guaranteed to be completed. The algorithm thus completes in $O(2B \cdot R \cdot \log_{1/\epsilon} n)$ rounds, and since $B = O(1)$, we obtain the stated $O(R \log n)$-round algorithm.

Inconsistent views and optimistic computation. The above intuitive algorithm relies on the nodes knowing, in every step, which k tasks remain incomplete. However, in our model, such perfect knowledge is unavailable. Although the network is fully connected and nodes may communicate in every round, this only provides partial information about failures. In particular, if a node crashes in

round r, some nodes may learn of the crash at the end of round r, while others learn of it only in round $r+1$. More importantly, discrepancies in the nodes' views of task completion can be substantial. For example, suppose nodes report to the entire network which tasks they have completed and then crash during this reporting step. In this case, only a subset of the nodes receive these reports, leading to disagreement across the network regarding which tasks remain incomplete. We note that approaches that attempt to redistribute tasks among non-crashed nodes (e.g., [25, 26, 6]) typically rely on all nodes sharing a consistent global view.

One possible solution is to run a crash-resilient consensus algorithm after every step to synchronize views. However, this approach is prohibitively expensive in terms of round complexity, adding $\Omega(n)$ rounds in the worst case. Instead, we take a different approach.

We execute the load balancing covering family–based approach described above without knowing the exact number k of incomplete tasks and without requiring full agreement on which tasks have been completed. To address the issue of inconsistent views, we first redefine the notion of task completion. A task is said to be *fully verified* if at least one node has completed it and *all* non-crashed nodes have been informed that it is complete. With this re-definition, the set of tasks that a given node knows to be complete may be a strict superset of the set of fully verified tasks, since some completion notifications may not have reached all nodes. Nevertheless, this suffices for our purposes. The properties of the load balancing covering family guarantee that each node is assigned approximately B tasks among the k tasks it believes to be incomplete (which, as noted above, form a superset of the not-yet fully verified tasks). In particular, since property (2) of the load balancing covering family holds for *any* choice of k subsets, this approach remains valid even when different nodes maintain different views of which k tasks are incomplete. After completing its assigned tasks, the node informs all other nodes that *all tasks* assigned to it have been completed. Thus, unless the node crashes during this process, this report makes all of these tasks fully verified regardless of the node's prior knowledge about their status.

It remains to explain how the algorithm operates without consistent knowledge of k. Our algorithm proceeds by *guessing* the number k of not-yet fully verified tasks up to a factor of ϵ and optimistically attempting to fully verify them. In each step, the guessed value decreases by a factor of ϵ, until it reaches a constant after $O(\log n)$ steps. For any step in which the guess is sufficiently accurate (i.e., ϵ-close to the real value), the network makes substantial progress by fully verifying at least ϵk tasks. This guarantees that the true number of not-yet fully verified tasks is always upper bounded by the algorithm's current guess. Moreover, whenever the true number approaches this upper bound, it necessarily lies within the range that ensures progress.

1.2 Application: Robust Congested-Clique Simulation

In recent work, Censor-Hillel, et al. [6] presented a general method for simulating any congested-clique computation on a network of size n in the presence of up to αn crash faults. Given an algorithm $\mathcal{A}$ with round complexity T, their approach converts $\mathcal{A}$ into an equivalent logic circuit C, which is then simulated by the network in a gate-by-gate manner. Specifically, the gates in the lowest uncomputed layer of C are distributed among the nodes, which compute them and *store* the resulting output in the network using a locally decodable error-correction code (LDC). Note that, in order to distribute the gates in a consistent manner, all nodes must agree on the set of non-crashed nodes. To this end, the simulation in [6] assumes *clean crashes*, in which nodes may crash only at the beginning of a round (before sending any messages), but never in the middle of a round.

To handle crashes, their simulation employs a doubling strategy: whenever a node crashes, *two* other nodes take over the gates assigned to the crashed node. Moreover, a node may fail to compute a gate if some of the nodes holding (coded parts of) its input values have crashed. In this case as well, the doubling strategy is applied, with the node doubling its attempts to compute the gate by contacting additional subsets of nodes that store the same information. The analysis in [6] shows that each doubling loop succeeds after $O(\log n)$ iterations for any crash pattern with at most αn crashes, for any $\alpha < 1$. As a result, the overall simulation requires $O(T^2 \cdot \log^2 n \cdot q)$ rounds: the circuit C has T^2 layers of gates, each layer incurs an $O(\log^2 n)$ overhead due to the doubling strategy, and $q = 2^{O(\sqrt{\log n} \log\log n)}$ denotes the round complexity required to decode information stored using the LDC without causing congestion.

An immediate application of our task completion algorithm to the approach of [6] is to view each gate as a task. On the surface of it, sequentially applying the task completion algorithm T^2 times then yields a simulation with the same asymptotic round complexity of [6], but without relying on the clean-crash assumption made in [6].

We can do even better. Rather than converting $\mathcal{A}$ into a circuit and simulating it gate by gate, we adopt a more direct approach that simulates $\mathcal{A}$ round by round using our algorithm for the task completion problem. Consider a single round of $\mathcal{A}$, in which the n nodes each send one message to each of their neighbors. We view this round as consisting of n tasks, where each node has the task of obtaining all messages designated to it during that round. Once all these tasks are completed, we can conclude that the round of $\mathcal{A}$ has been correctly simulated.

However, this translation into tasks does not directly satisfy the assumptions required by our task-completion algorithm. In particular, in this setting a node may fail to obtain all of its designated messages (and thus, fail to complete its task) even if it *does not* crash itself. That is, the task is not *executor-safe*. Such a failure may occur when some of the nodes responsible for sending these messages crash before delivering them. By contrast, in the standard task completion problem, the only reason a task may fail to be completed is that the node assigned to execute it crashes. This mismatch requires additional care in adapting the task completion algorithm to the round-by-round simulation setting.

Our solution addresses this issue by decomposing the required communication into two *nested* subtasks, each of which is carried out using our task-completion algorithm. The first subtask, which we refer to as the *inner* task, assumes that all messages sent by node i in the r-th simulated round, $M_i(r)$, are already stored in the network using a standard error-correction code (ECC) of block length n. In this encoding, each node holds a single symbol of the codeword $C_i(r) = \mathrm{Enc}(M_i(r))$, and the original messages $M_i(r)$ can be recovered even if up to αn symbols of $C_i(r)$ are missing; see Section 2.3. The i-th inner task consists of decoding the codeword $C_i(r)$ to obtain all the n messages sent by node i in the r-th simulated round of $\mathcal{A}$, and then delivering to each node j the message designated to it by node i.

This inner task alone is insufficient if node j has already crashed: in that case, it is unclear where the message designated to j should be delivered. To resolve this issue, we introduce a second subtask we call the *outer* task. The j-th outer task is to *act as node j* in the simulation of $\mathcal{A}$, that is, to collect all messages designated to j and generate the messages that j needs to send in the next round, $M_j(r+1)$. Specifically, once a node decides to act as the node j in $\mathcal{A}$ in the outer task, all nodes initiate, in a nested manner, a task-completion instance for the inner task. The inner task guarantees that the node acting as j receives all n messages intended for j in round r of $\mathcal{A}$, which we denote by $S_j(r)$. The node then stores $S_j(r)$ in the network using an ECC (for technical reasons

explained later). Finally, the node computes all messages that j should send in the next round of $\mathcal{A}$, $M_j(r+1)$, encodes these messages using an ECC, and stores the resulting codeword in the network by sending one symbol to each node. This codeword subsequently serves as the input for simulating the next round of $\mathcal{A}$.

We note that both the inner and outer tasks are idempotent. Specifically, if multiple nodes complete the same task and send messages or store codewords in the network, they transmit or store identical information. As a result, no inconsistency is introduced by redundant executions of the same task. Moreover, both tasks are executor-safe. In particular, although the node simulating node j in the outer task relies on other nodes to deliver the required messages, it does not depend on any specific helper node. Instead, message delivery is carried out through the task-completion mechanism of the inner task in a fault-safe manner. Consequently, if any node responsible for delivering messages crashes, it is automatically replaced by other nodes via task completion. This ensures that the outer task completes successfully as long as the node simulating node j itself does not crash.

As described above, the outer and inner tasks are executed in a nested manner. First, each node participates in the outer task algorithm and obtains an identity j to simulate. The node then informs all other nodes that it is simulating identity j (multiple such nodes may exist). Once all nodes have announced the identities they simulate, the network initiates an instance of the inner task, whose goal is to deliver messages to the simulated identities. This phase takes $O(\log n)$ rounds using our task completion algorithm for the inner task, and by its conclusion, all simulated identities hold all messages required for that round. The outer task then proceeds to its next step, during which non-crashed nodes assume identities that have not yet been simulated. After $O(\log n)$ steps of the outer task, all identities will have been simulated. Consequently, simulating a single round of $\mathcal{A}$ requires $O(\log^2 n)$ rounds.

Unfortunately, this does not yet imply an $O(T \log^2 n)$-round simulation of $\mathcal{A}$. The difficulty arises from the fact that a node simulating identity j cannot generate the outgoing messages of j in round r of $\mathcal{A}$ solely from the messages that j received in round $r-1$. Rather, to compute these messages, the simulating node must have access to the complete *state* of node j at the beginning of round r. There are several ways to address this issue. If the state (i.e., memory) of node j is short, e.g., of size $O(n \log n)$, then storing this state at each round together with the codeword $C_j(r)$ is a viable option, yielding an overall $O(T \log^2 n)$-round simulation of $\mathcal{A}$. However, in the general case, storing and decoding the full state is prohibitively expensive. A more general solution is for the node simulating identity j to first reconstruct the state that j would have at the beginning of round r. This can be achieved by obtaining the information $S_j(1), ..., S_j(r-1)$, which correspond to the messages that j received in all rounds prior to r. As a result, simulating round r of $\mathcal{A}$ requires $O(r + \log n)$ rounds to complete the outer task. Overall, the obtained round complexity is $O(T^2 \log n + T \log^2 n)$.

1.3 Other Related Work

Several extensions of the task completion problem have appeared in the literature. Randomized algorithms were introduced in [14, 13], where randomization reduces the total work to $n \log^2 n$ against an arbitrary adaptive adversary with $f < n$ failures, and to $O(n \log^* n)$ against a weakly adaptive adversary that selects f nodes prior to the start of execution and may crash only these selected nodes. Variants of task completion with crashing nodes that can later restart were studied in [11]. Algorithms tolerating Byzantine faults rather than crash faults were presented in [22].

A variant of the task completion problem in the PRAM shared-memory model is known as the *write-all* problem [31]. In this setting, n memory cells are initially set to 0, and n processors are required to write the value 1 to all cells [1, 33, 4]. A common technique in this line of work is to assign each processor a predetermined permutation of $\{1, \ldots, n\}$, which specifies the order in which it attempts to write to the memory cells. A processor skips any cell that it already knows has been written. Optimizing such permutations [42, 36] amounts to efficiently finding a collection of permutations that minimizes the total number of write attempts. In this sense, our combinatorial load balancing covering family can be viewed as an extension of this approach: rather than fixing a single permutation, it effectively induces randomized task orders with desirable properties, such as sufficient redundancy and low congestion. We note that the above works typically assume a model in which up to $n-1$ processors may fail. In this extreme setting, the last remaining processor must eventually perform all tasks, which motivates the use of permutations over the entire task set. Simulations of arbitrary PRAM algorithms using write-all primitives were presented in [48, 34, 32].

The congested clique model [40] has been extensively studied in the literature. A large body of work explores a wide range of algorithmic questions and graph-theoretic problems, including minimum spanning tree computation [40, 43, 28, 27, 35, 30], routing [39], graph coloring [45, 46, 10, 3, 17, 16], subgraph detection and enumeration [20, 29, 44, 23, 7, 8, 5], and many others. Despite this extensive body of work, significantly less is known about fault tolerance in the congested clique model. Only a limited number of recent works address resilience to failures in this setting [2, 38, 37, 15, 24, 6, 9].

1.4 Paper Outline

After presenting the model and preliminaries in Section 2, we define in Section 3 the notion of a *load balancing covering family* and prove its existence for suitable parameters. In Section 4 we present a deterministic task completion algorithm based on these coverings, prove its correctness, and analyze its round complexity. A lower bound on the round complexity of any deterministic task-completion algorithm is given in Section B. Finally, in Section 5 we apply the task completion framework to obtain a crash-resilient simulation of arbitrary T-round congested-clique algorithms, with a total complexity of $O(T^2 \log n + T \log^2 n)$ rounds.

2 Preliminaries

For an integer $n \geq 1$, we let $[n]$ denote the set $\{1, 2, \ldots, n\}$. All logarithms are taken to base 2 unless otherwise mentioned. A string S over an alphabet Σ is a consecutive sequence of 0 or more letters $S = s_1, s_2, \ldots$ where each letter $s_i \in \Sigma$. The length of the string is denoted $|S|$. For $1 \leq i \leq |S|$, we let $S[i] = s_i$ denote the i-th letter in S.

2.1 Network, Crash-faults, and the Congested Clique Model

We assume a fully connected network $G = (V, E)$ of size $|V| = n$, where all nodes know the identities of all their neighbors. For the ease of notation we identify the set of nodes V with $[n]$. Communication follows the standard synchronous CONGEST model, namely, communication occurs in synchronous rounds, where in each round, each node can send $O(\log n)$ bits to each one of its neighbors.

A node may crash at any time of the execution. If a node v crashes during some round, then an arbitrary subset of its outgoing messages may fail to be sent in that round and v cannot send messages in any future round. If a message fails or is not sent at all, the recipient node obtains the

symbol $\perp$. Note that if the recipient was expecting a message from v, the reception of a $\perp$ indicates that the sender v has crashed. We assume that at most αn nodes may crash over the entire execution of the protocol, for a fixed and known constant parameter $\alpha \in [0, 1)$.

As noted in prior work [6, 9], special care is required to prevent the loss of nodes' inputs if they crash before the computation begins. Existing approaches address this issue either by guaranteeing that no node crashes during the first few rounds [9], or by modifying the model so that inputs are provided in encoded form and stored by the network [6]. We adopt the latter approach, as it yields better composability. In our model, the input of each node has length $O(n \log n)$ bits and is encoded using an error-correction code (see Section 5.1 and Section 2.3 below). Each node holds a single symbol of each encoded input. Consequently, even if a node crashes before the computation starts, its input can still be recovered by querying the corresponding codeword symbols. Similarly, we require that the output of each node be encoded and stored in the same manner. Then, composing two algorithms, where the output of the former serves as the input to the latter, is straightforward.

2.2 The Task Completion Problem

In the task completion problem, we have M abstract tasks with unique labels $[M]$. Initially, all tasks are defined as *incomplete*. At the start of any round, each node v may decide to complete *any* task of its choice. The completion of a task takes $R \geq 1$ rounds, after which the task is called *completed*. While tasks represent arbitrary activities (even outside the prescribed model), we only consider tasks that satisfy the following three properties. Tasks must be *independent*: the completion of any task does not affect any other task, and any task can be executed concurrently with any other task; further, up to n tasks can be executed concurrently. The tasks must also be *idempotent*: each task can be executed one or more times by one or more nodes and will produce the same final result; in particular, completing an already completed task does not change its status or compromise its validity. Finally, tasks must be *executor-safe*: a task completes successfully whenever the node executing it does not crash, regardless of failures of other nodes.

We say that a deterministic algorithm solves the task completion problem if, once it finishes running, every task has been completed, regardless of how crashes occur during its execution, provided that there are at most αn such crashes.

2.3 Error Correction Codes

For an alphabet Σ, the Hamming distance of two strings $x, y \in (\Sigma \cup \perp)^*$ of the same length, i.e., $|x| = |y|$, is the number of indices for which x and y differ and is denoted by $\text{Hamm}(x, y) = |\{i \mid x[i] \neq y[i]\}|$. For two strings $x \in \Sigma^*$, $y \in (\Sigma \cup \{\perp\})^*$ and a value $c \in (0, 1)$, we say that y can be obtained by a c-fraction of erasures from x if $|x| = |y|$, and for all $i \in [|x|]$, it holds that either $x[i] = y[i]$ or $y[i] = \perp$, where the latter case happens at most $c|x|$ times. An index i in which $y[i] = \perp$ is called an erasure.

For a prime power p, we denote by $\mathbb{F}_p$ the finite field of size p. An *error correction code* is a mapping $\text{Enc} : \mathbb{F}_p^K \to \mathbb{F}_p^N$ that takes K symbols of the alphabet $\mathbb{F}_p$ into N symbols of the alphabet $\mathbb{F}_p$. The value N is called the block length of the code. The ratio $\rho = K/N$ is called the *rate* of the code. The *relative distance* of a code is the minimal normalized Hamming distance between any two codewords, denoted $\delta = \min_{m \neq m'} \frac{1}{N} \text{Hamm}(\text{Enc}(m), \text{Enc}(m'))$. The following claim is well known,

Claim 2.1 (MDS Erasure Correction Codes [47])**.** *For any $N > K > 1$, there exists a code $\mathbb{F}_p^K \to \mathbb{F}_p^N$, where $p \geq N$, with relative distance $\delta = (N - K + 1)/N$. Such a code can correct up to $N - K$ erasures.*

As a corollary, for any $\alpha < 1$ and a sufficiently large N, by choosing the right parameters, we can have a code with block length n and relative distance $\delta > \alpha$, that can correct α-fraction of erasures. Further, each symbol in such a code is of length $O(\log N)$ bits.

3 Load Balancing Covering Sets

We begin by defining a (k, B, ϵ)-load balancing covering family and then prove its existence for appropriate choices of the parameters.

Definition 1. *Let $1 \leq m \leq n$ be two fixed integers. For any integer parameters $B, k \geq 1$, and value $\epsilon \in (0, 1)$, we say that a family $A_1, \ldots, A_m \subseteq [n]$ is (k, B, ϵ)-load balancing covering family if the following conditions are satisfied:*

1. *$\frac{nB}{2k} \leq |A_i| \leq \frac{2nB}{k}$.*
2. *For any k sets $A_{i_1}, \ldots, A_{i_k}$ of this family, there exist a set $J \subseteq [n]$ of size $|J| \geq (1 - \epsilon)n$, such that for all $j \in J$,*

$$(1 - \epsilon)B \leq |\{\ell \in [k] \mid j \in A_{i_\ell}\}| \leq (1 + \epsilon)B.$$

Lemma 3.1. *Let $\epsilon \in (0, 1)$ be a constant. Let $c = \epsilon^3/8$, and $B = \lceil 72/c \rceil$. For any sufficiently large n, any integer $m \leq cn$ and any $B \leq k \leq m$, there exists a (k, B, ϵ)-load balancing covering family.*

Proof. We prove the existence of such a family using the probabilistic method. For each set $A \in \{A_1, \ldots, A_{2m}\}$ we construct A by including in it each element $j \in [n]$ with probability B/k, independently across elements and sets. We show that with good probability, most of these sets are of size $|A_i| \in \left[\frac{nB}{2k}, \frac{2nB}{k}\right]$, and that Definition 1(2) holds for any choice of k sets from these $2m$ sets. We then remove m sets so that all remaining sets satisfy Definition 1(1), while the property in Definition 1(2) endures.

First, we prove that Definition 1(1) holds for more than half the sets, with probability strictly larger than 1/2. By linearity of expectations, for any $i \in [m]$ it holds that $\mathbb{E}[|A_i|] = nB/k$. Then, it follows that

$$\Pr\left(\frac{nB}{2k} \leq |A_i| \leq \frac{2nB}{k}\right) \geq 1 - 2e^{-nB/(12k)} \geq 0.99, \tag{1}$$

where the first inequality follows by Chernoff's inequality (Theorem A.1(3), with $\delta = 1/2$), and the next inequality by recalling that $k \leq m < n$, and by taking B to be sufficiently large, e.g., $B > 72/c$. Let M be the number of sets that are of size $\in [\frac{nB}{2k}, \frac{2nB}{k}]$. We note that by Equation (1), we have $\mathbb{E}[M] \geq 0.99 \cdot 2m$. Thus, the probability that $M \geq m$ is at least

$$\Pr(M \geq m) = 1 - \Pr(M < m) \geq 1 - \Pr(M < (1 - 0.98)\mathbb{E}[M]) \geq 1 - e^{-1.98m \cdot (0.98)^2/2},$$

where the last step follows by Chernoff's inequality (Theorem A.1(2), with $\delta = 0.98$). For $m \geq 1$, this is above $0.6 > 1/2$.

Next, we show that Definition 1(2) holds with probability strictly larger than $1/2$, for any k sets out of all possible $2m$ sets. Fix k sets $A_{i_1}, \ldots, A_{i_k}$ from $\{A_1, \ldots, A_{2m}\}$ and let $I = \{i_1, \ldots, i_k\}$. For any $j \in [n]$ we define the *load* of j in I to be

$$\mathsf{load}(j, I) = |\{\ell \in I \mid j \in A_\ell\}|, \tag{2}$$

namely, the number of subsets $\{A_i\}_{i \in I}$ that include the element j. Let $B(j, I)$ be the expectation of the load of j in I,

$$B(j, I) \coloneqq \mathbb{E}[\mathsf{load}(j, I)] = k \cdot (B/k) = B.$$

Note that this expectation value does not depend on the specific values of $i_1, \ldots, i_k$ and j (but it does depend on $k = |I|$); thus, $B(j, I) = B$ for all $j \in [n]$ and I's of size k. By Chernoff's inequality (Theorem A.1(3)), for any fixed $j \in [n]$,

$$\Pr(|\mathsf{load}(j, I) - B| > \epsilon B) < 2e^{-\epsilon^2 B/3} < \epsilon/2. \tag{3}$$

For j and I as above, set the indicator $\mathsf{bad}(j)$ to be the event where the index j has a load which is ϵ-far from its expected value, i.e., the event

$$\mathsf{load}(j, I) \notin [(1-\epsilon)B, (1+\epsilon)B].$$

Finally, set J to be all the indices $j \in [n]$ which are not bad, namely, $J \coloneqq [n] \setminus \{j \mid \mathsf{bad}(j)\}$. To complete the proof, we need to show that $|J| \geq (1-\epsilon)n$; let us then bound the probability that $|J| < (1-\epsilon)n$. For a fixed $j \in [n]$, we have by Equation (3) that

$$\Pr(j \in J) = \Pr(|\mathsf{load}(j, I) - B| \leq \epsilon B) \geq 1 - \epsilon/2.$$

By linearity of expectation, $\mathbb{E}[|J|] = \sum_{j \in [n]} \Pr(j \in J) \geq (1-\epsilon/2)n$. For $j \in [n]$, consider the events that $j \in J$ and note that these events are mutually independent for $j' \neq j$. Then, by Chernoff's inequality (Theorem A.1(2)), we get

$$\Pr(|J| < (1-\epsilon)n) \leq \Pr\left(|J| < (1-\epsilon/2)^2 n\right) \leq e^{-(\epsilon/2)^2(1-\epsilon/2)n/2}.$$

By taking union bound over all the choices of $I \subset [2m]$ of size $|I| = k$, i.e., all possible k subsets of the family, we get that they all satisfy the condition simultaneously with probability at least $1 - e^{-(\epsilon/2)^2(1-\epsilon/2)n}\binom{2m}{k} > 1/2$, where the inequality follows by the fact that $m \leq cn$ with a sufficiently small $c > 0$ with respect to ϵ. Namely, $\binom{2m}{k} \leq 2^{2m} \leq 2^{2cn} < e^{(\epsilon/2)^2(1-\epsilon/2)n-1}$.

Summing it all up, by a union bound, the probability that a randomly chosen $\{A_1, \ldots, A_{2m}\}$ does not satisfy Definition 1(2) or has less than m sets of size satisfying Definition 1(1), is *strictly smaller* than $1/2 + 1/2 = 1$. Hence, there exists a load balancing covering family with the desired parameters. Note that the proof holds for any sufficiently large $B = \Omega(1/\epsilon^3)$, that is bounded above by $B \leq k \leq m$. □

4 A Deterministic Task-Completion Algorithm

In this section we present our algorithm for the task completion problem, depicted in Algorithm 1, and prove the following.

Theorem 4.1. *Assuming that completing a single task requires $R \geq 1$ rounds, for any $\alpha \in [0, 1)$ there exists a deterministic algorithm that solves the M-task completion problem in a congested clique of size n in the presence of up to αn crashes, using $O(R\lceil M/n \rceil \log n)$ rounds.*

4.1 Algorithm Description

Let $\epsilon = \epsilon(\alpha) > 0$ be a sufficiently small constant relative to α, and let $c = c(\epsilon) > 0$ be a sufficiently small constant relative to ϵ. In this section, we assume that the number of tasks is $m = cn$. However, any number M of tasks can be handled by partitioning them into sets of size at most cn and completing each set sequentially (see Section 4.3).

Algorithm 1 Iterative Task Completion for n nodes and $m = cn$ tasks (code for node v)

Inputs: The maximal fault-tolerance constant $\alpha > 0$, the number of parties n, the number of tasks $m = cn$. Parameters B, ϵ, c to be specified later (as a function of α, n)

$C_{1,v} \leftarrow \emptyset$
for $i = 1, 2, \ldots, \Theta(\log n)$ **do**
  $k_i \leftarrow \lceil (1-\epsilon)^{i-1} m \rceil$
  if $1 \le k_i \le 2B$ **then**
    $T_{i,v} \leftarrow [m]$ ▷ give all tasks to all nodes
  else
    Let $A_1, \ldots, A_m$ be a (k_i, B, ϵ)-load balancing covering family, agreed upon by all nodes.
    $T_{i,v} \leftarrow \{\, t \in [m] \mid v \in A_t \,\}$ ▷ $T_{i,v}$ depends only on (i, v)
  end if
  $U_{i,v} \leftarrow T_{i,v} \setminus C_{i,v}$ ▷ only tasks not yet known to be completed
  For $2B \cdot R$ rounds, complete tasks in $U_{i,v}$ by lexicographic order; idle if no tasks to complete.
  if v has completed all tasks in $U_{i,v}$ **then**
    v broadcasts $s_{i,v} = 1$ to all nodes.
  else
    v broadcasts $s_{i,v} = 0$.
  end if
  Receive $s_{i,u}$ from all nodes u.
  ▷ $s_{i,u} = \perp$ if u crashed before sending to v
  $C_{i+1,v} \leftarrow C_{i,v} \cup \bigcup_{\substack{u \in [n]: \\ s_{i,u}=1}} T_{i,u}$
end for

Our task-completion algorithm, Algorithm 1, consists of $\Theta(\log n)$ iterations. In iteration i, each node v is assigned a set of tasks $T_{i,v} \subseteq [m]$, deterministically generated via a load balancing covering family as a function only of i and v; these sets are therefore globally known to all nodes (we explain the construction in detail below). The goal of node v is to complete all incomplete tasks in $T_{i,v}$. Throughout the algorithm, each node maintains a list $C_{i,v}$ of tasks that v knows to be completed at the start of iteration i, either because v completed them in a previous iteration or because some other node completed them and reported this to v. In iteration i, node v attempts to complete all tasks in $T_{i,v} \setminus C_{i,v}$, one after the other. Our analysis shows that, for all but a small fraction of the nodes, this set of potentially incomplete tasks has size at most $2B$, for a parameter $B = B(\epsilon)$. Hence, after $2B \cdot R$ rounds, most nodes succeed in completing *all* of their assigned tasks unless they crash during these $2BR$ rounds. If node v does not crash during these rounds, and $|T_{i,v} \setminus C_{i,v}| < 2B$,

then, upon completing all these tasks, v broadcasts a success bit $s_{i,v} = 1$; otherwise, it broadcasts $s_{i,v} = 0$ to indicate that it did not complete all of its assigned tasks.

Initially, $C_{1,v} = \emptyset$. At the end of iteration i, node v updates $C_{i+1,v}$ to include all tasks in $C_{i,v}$, as well as all tasks in $T_{i,u}$ for every $u \in [n]$ such that $s_{i,u} = 1$. To do so, node v must know the sets $T_{i,u}$ for all $u \in [n]$, which indeed holds, as we explain next.

In iteration i, we set $k_i = \lceil (1-\epsilon)^{i-1} m \rceil$. If $1 \leq k_i \leq 2B$, all nodes set $T_{i,v} = [m]$. Otherwise, each node deterministically constructs a (k_i, B, ϵ)-load balancing covering family using Lemma 3.1 with parameters $n = n$, $m = m$, $k = k_i$, and $\epsilon = \epsilon$ (the left-hand side corresponds to the lemma's parameters, and the right-hand side to the task-completion parameters). This construction yields a family of sets $A_1, \ldots, A_m \subseteq [n]$, where each set A_t consists of all nodes assigned to task t. In other words, if $v \in A_t$, then $t \in T_{i,v}$. Since this construction is deterministic and depends only on k_i, B, and ϵ, all nodes share the same load balancing covering family and can therefore compute the sets $T_{i,v}$ for all $v \in [n]$.

The properties of the load balancing covering family guarantee that almost all nodes are assigned with $O(B)$ tasks that still need completion, and that each task that needs completion is assigned to $\approx nB/k_i$ different nodes. These two properties guarantee that enough progress is made in each iteration, namely, at least ϵ-fraction of the tasks that need completion will be completed and all non-crashed nodes will be aware of this. After $O_\epsilon(\log n)$ iterations, all tasks will be completed.

4.2 Analysis

We now prove both the correctness of our algorithm and its round complexity. For a task $t \in [m]$, we say that t is *fully verified* at the start of iteration i if $t \in C_{i,v}$ for all nodes that have not crashed by that time. Let N_i denote the number of tasks that are not fully verified at the start of iteration i. The following is immediate from the definition.

Claim 4.2. *For any iteration i and any non-crashed v, $|C_{i,v}| \geq m - N_i$*

Proof. If there are N_i tasks that are not fully verified, the other $m - N_i$ tasks are fully verified and in particular reside in $C_{i,v}$ for any non-crashed v. □

The main part of this section is the proof of the following lemma.

Lemma 4.3.

1. *When Algorithm 1 terminates, all tasks are fully verified.*
2. *Algorithm 1 terminates after $O(R \cdot B \log n)$ rounds.*

We start by proving the first part of Lemma 4.3. Towards this goal we show that the variable $C_{i,v}$ that v holds indeed indicates only completed tasks. We then show that the number N_i of non-fully-verified tasks decreases to zero as the algorithm progresses. Once we reach an iteration i for which $N_i = 0$, all tasks reside in all the $C_{i,v}$'s of the non-crashed nodes, which in particular means that all tasks were completed.

Lemma 4.4. *If $t \in C_{i,v}$ for some v, then the task t has been completed before start of iteration i.*

Proof. Proof by induction on i. For $i = 1$, the claim holds vacuously, since $C_{i,v} = \emptyset$.

Now, assume the claim holds at the start of some iteration $i \geq 1$, and consider the start of the $(i+1)$-st iteration. Recall that v sets its new $C_{i+1,v}$ by

$$C_{i+1,v} \leftarrow C_{i,v} \ \cup \bigcup_{u \in [n]:\ s_{i,u}=1} T_{i,u}.$$

Consider a task $t \in C_{i+1,v}$. If $t \in C_{i,v}$, then by the induction hypothesis it was completed before iteration i. Next, consider a task $t \in \bigcup_{u \in [n]:\, s_{i,u}=1} T_{i,u}$. In this case, there exists some node u such that $s_{i,u} = 1$ and $t \in T_{i,u}$. By the condition in line 12, node u has completed all tasks in $T_{i,u} \setminus C_{i,u}$. Consequently, either t was completed by u during iteration i, or $t \in C_{i,u}$, in which case it was completed before iteration i by the induction hypothesis. We therefore conclude that every task in $C_{i+1,v}$ has been completed by the start of iteration $i+1$. □

Next, we show that for every iteration i, the value k_i used by the algorithm upper-bounds the number of tasks that are not fully verified, namely N_i.

Lemma 4.5. *For any iteration i, $N_i \leq k_i$.*

Proof. The lemma follows (by induction) directly from the following two claims, that hold for any iteration i: (a) $N_{i+1} \leq N_i \leq m$, and (b) if $(1-\epsilon)k_i \leq N_i \leq k_i$, then $N_{i+1} \leq (1-\epsilon)N_i$. Indeed, for $i = 1$ we have $N_1 = k_1 = m$. For the induction step, assume that $N_i \leq k_i$ and recall that $k_{i+1} = \lceil (1-\epsilon)^i m \rceil$. If $N_i \leq k_{i+1}$, the lemma holds from claim (a); otherwise, the conditions of claim (b) hold, and it follows that $N_{i+1} \leq \lfloor (1-\epsilon)N_i \rfloor \leq \lfloor (1-\epsilon)k_i \rfloor = \lfloor (1-\epsilon)\lceil (1-\epsilon)^{i-1} m \rceil \rfloor \leq \lceil (1-\epsilon)^i m \rceil = k_{i+1}$, where the first inequality holds since N_{i+1} is an integer, hence flooring the right-hand-side is allowed, and the last inequality holds for any $\epsilon \in (0,1)$ and integers i, m (see Lemma A.2). Let us now prove the above two claims.

The proof of (a) is immediate: the number of tasks that are not fully verified is clearly bounded by the total number of tasks m, and monotonically decreases as tasks are being completed. We continue to proving item (b); let us split the proof into two sub-cases.

If $k_i \leq 2B$, then (b) follows since, in this case, all non-crashed nodes v are assigned with all the tasks $[m]$. Consider any node v' that is still non-crashed at the end of iteration i. Note that $|C_{i,v'}| \geq m - N_i \geq m - k_i$, which follows from Claim 4.2 and from the conditions of part (b), i.e., $N_i \leq k_i$. During this iteration, v' sets $T_{i,v'} = [m]$, and $U_{i,v'} = [m] \setminus C_{i,v'}$. By the above, $|U_{i,v'}| \leq k_i \leq 2B$, hence in line 11, v' succeeds in completing *all* the tasks in $U_{i,v'}$ and sets $C_{i+1,v'} = [m]$. Since the above applies to all non-crashed nodes, this makes all the tasks fully verified and $N_{i+1} = 0$.

Let us now consider the case where $k_i > 2B$. Let $I \subseteq [m]$ be the set of the N_i tasks that are not fully verified at the beginning of iteration i, and let $I' \subseteq [m]$ be the (lexicographically minimal) set of $|I'| = k_i$ tasks such that $I \subseteq I'$. Let $A_1, \ldots, A_m$ be the (k_i, B, ϵ)-load balancing covering family from line 7, and let J_i be the set guaranteed by Definition 1(2) for the k_i subsets $\{A_{i'}\}_{i' \in I'}$.

For any task t that is not fully verified at the start of iteration i, we say that a non-crashed node v *attempts to fully verify* t in iteration i if $t \in T_{i,v}$. We claim that for any such task t, if there exists a node $v \in J_i$ that attempts to fully verify t and does not crash during iteration i, then t becomes fully verified by the end of the iteration. To see this, fix such a node v. Any task $t' \notin I'$ is already fully verified and therefore belongs to $C_{i,v}$. Recall that $\mathsf{load}(j, I') \triangleq |\{\, i' \in I' \mid j \in A_{i'} \,\}|$, and that the load balancing covering family guarantees that for any $j \in J_i$, $\mathsf{load}(j, I') \leq (1+\epsilon)B \leq 2B$ (Definition 1(2)). It follows that $|U_{i,v}| = |T_{i,v} \setminus C_{i,v}| \leq \mathsf{load}(v, I') \leq 2B$. Hence, node v can complete

all tasks in $U_{i,v}$ within $2BR$ rounds (line 11). Since $t \in T_{i,v}$, task t is completed during this phase (if wasn't already complete). Node v then broadcasts $s_{i,v} = 1$, which makes t fully verified once this message is received by all non-crashed nodes.

To complete the proof of claim (b), we need to show that for at least ϵk_i tasks that are not fully verified, there exists a node in J_i that does not crash during iteration i and attempts (and thus succeeds) to complete them. We now count the total number of attempts made by all nodes in J_i to verify some non-fully-verified task, i.e.,

$$\#\text{attempts} := \sum_{\substack{t \text{ is not fully verified} \\ \text{at the start of iteration } i}} |\{v \in J_i \mid t \in T_{i,v}\}|.$$

We first prove that

$$\#\text{attempts} \geq (1 - 4\epsilon)nB. \tag{4}$$

Recall that $(1 - \epsilon)k_i \leq N_i \leq k_i$, and that $I' \subseteq [m]$ contains exactly k_i tasks that includes all the N_i non-fully-verified tasks. Note that the number of fully verified tasks in I' never exceeds ϵk_i. By Definition 1(1), the total number of nodes that attempt to verify in iteration i a task $t \in I'$ that is already fully verified is at most $\epsilon k_i \cdot 2nB/k_i$; this bound also applies to their number within J_i. Further, each node in J_i is assigned with at most $(1 + \epsilon)B$ different tasks, so all these attempts complete during the $2BR$ rounds of line 11. It follows that nodes in J_i make at least the following number of attempts to verify tasks that are not fully verified:

$$\#\text{attempts} \geq |J_i|(1 - \epsilon)B - 2\epsilon nB \geq ((1 - \epsilon)^2 - 2\epsilon)nB,$$

and Equation (4) follows.

Moreover, the set J_i contains at most αn crashed nodes (including nodes that crash during iteration i), which correspond to at most $(1 + \epsilon)\alpha nB$ attempts that were counted in Equation (4) but may not actually occur. Summing up, at least $(1 - 4\epsilon - (1 + \epsilon)\alpha)nB$ attempts are made by non-crashed nodes in J_i to verify tasks that are not fully verified, and these attempts succeed in iteration i.

Since each non-fully-verified task is associated with at most $2nB/k_i$ nodes that may attempt to verify it (Definition 1(1)), a pigeonhole argument implies that the number of non-fully-verified tasks at the start of iteration i that become fully verified by the end of the iteration satisfies

$$N_i - N_{i+1} \geq \frac{(1 - 4\epsilon - (1 + \epsilon)\alpha)nB}{2nB/k_i} = \frac{1 - 4\epsilon - (1 + \epsilon)\alpha}{2} k_i.$$

We choose $\epsilon > 0$ to be a sufficiently small constant such that $(1 - 4\epsilon - (1 + \epsilon)\alpha)/2 \geq \epsilon$. It follows that at least ϵk_i tasks from N_i become fully verified by the end of iteration i. Recalling that $N_i \leq k_i$, it follows that at least an ϵ-fraction of the non-fully-verified tasks become fully verified, $N_{i+1} \leq N_i - \epsilon k_i \leq (1 - \epsilon)N_i$, as required to complete the proof of claim (b). □

The above key lemma now allows us to complete the proof of Lemma 4.3.

Proof of Lemma 4.3: By Lemma 4.5, when $k_i < 1$, then all tasks are fully verified; by setting the main loop to run $\Theta(\log n)$ until $k_i < 1$, we guarantee that Lemma 4.3(1) holds at the end of the last iteration.[3] The above argument also proves Lemma 4.3(2): Algorithm 1 terminates after $\Theta(\log n)$ iterations, where each iteration takes $2BR + 1$ rounds (lines 11–17), totaling $\Theta(R \cdot B \log n)$ rounds.

[3]In fact, if $k_i \leq 2B$, then at the end of this iteration, $N_{i+1} = 0$ (see the proof of the first sub-case of claim (b) in Lemma 4.5), and the algorithm terminates in this iteration. Still, $\Theta(\log n)$ iterations are needed overall.

4.3 Completing the Proof of Theorem 4.1

By Lemma 4.3, Algorithm 1 correctly solves the m-task completion problem with $m < cn$ tasks in $\Theta(R \cdot B \log n)$ rounds, despite up to αn crashes. To solve the M-task completion problem, Algorithm 1 is executed $\lceil M/m \rceil$ times on batches of $m = cn$ tasks each, where c is the parameter given by Lemma 3.1 to obtain a (k, B, ϵ)-load balancing covering family. By choosing $B = \Theta(1/\epsilon^3)$, Lemma 3.1 guarantees the existence of a (k_i, B, ϵ)-load balancing covering family for any $k_i \leq m$ and any $B \leq k_i$. Under this choice, the M-task completion problem is solved using $\lceil M/m \rceil = O(\lceil M/n \rceil)$ sequential invocations of Algorithm 1, where each instance takes $\Theta(R \cdot B \log n) = \Theta(R \log n)$ rounds. This completes the proof of our main theorem, Theorem 4.1.

5 Robust Simulation of Congested Clique Algorithms

In this section, we show how to simulate any deterministic T-round Congested Clique algorithm $\mathcal{A}$, in the presence of up to αn crashes. In particular, we prove the following.

Theorem 5.1. *Let $\mathcal{A}$ be a deterministic congested-clique algorithm that runs for T rounds, where the input of each node has size $O(n \log n)$. Then, for any $\alpha \in [0, 1)$, there exists a congested-clique algorithm that simulates the computation of $\mathcal{A}$ in the presence of up to αn crashes and runs in $O(T^2 \log n + T \log^2 n)$ rounds.*

Before describing the algorithm, we briefly introduce and define several primitives and building blocks used in our simulation and establish some useful notation.

5.1 Storing and Retrieving Information in the Network

In this section, we describe several basic procedures that serve as building blocks for our simulation, namely NetStore and NetRetrieve, which are used to robustly maintain information in the network.

Error correction codes. To store and retrieve information from the network we utilize an error correction code (see Section 2.3). We remark that any code with constant rate and relative distance strictly greater than α suffices for our purposes, including codes over constant-size alphabets. However, for simplicity, we will use the MDS code given by Claim 2.1 with $N = n$ and $K = \rho n$ (an integer) for some constant rate $\rho < (1 - \alpha)$ that implies relative distance $\delta > \alpha + 1/n$. By Claim 2.1, the code can correctly decode any codeword that suffers up to αn erasures. To further simplify the usage of this code, we will assume n is a prime and use the alphabet $\mathbb{F}_n$; if n is not prime, a larger field can be used instead. Hence, we can write $\text{Enc} : [n]^{\rho n} \to [n]^n$ and $\text{Dec} : [n]^n \to [n]^{\rho n}$ as the encoding and decoding algorithms of the error correction code, respectively.

Storing information in the network. The NetStore procedure saves information in the network so that the information is recoverable even when up to αn nodes crash. The operation NetStore(S, I) is executed by a single node and takes two inputs: a string $S \in [n]^*$ and a key $I \in [\text{poly}(n)]$. The string S represents the information to be stored, while the key I serves as an index that enables referring to and retrieving this information at a later time. To store S in the network, the node splits S into parts of size ρn each, padding the last part with zeros if needed; denote these parts by $S_1, \ldots, S_{\lceil |S|/\rho n \rceil}$. The node then encodes each part S_j using an error correction

code to obtain the n-symbol codeword $\mathrm{Enc}(S_j)$. The node then sends one symbol of each codeword to each node, namely, for $\ell \in [n]$, the node sends $\mathrm{Enc}(S_j)[\ell]$ to node ℓ, along with the metadata I, and j. Once the node ℓ receives this information it saves it locally in a variable, which we denote $\mathrm{Stored}^{\ell}_{I,j}$.

Observation 5.2. *For $S \in [n]^*$, NetStore$(S, \cdot)$ takes $O(\lceil |S|/n \rceil)$ rounds.*

For the purposes of analysis, a NetStore operation is said to be *successful* if the executing node v does not crash before the end of the procedure. We also observe that the operation NetStore(S, I) is idempotent. Specifically, if multiple nodes hold the same pair (S, I) and execute NetStore(S, I), the information sent to each node in the network is identical. Consequently, the local variable $\mathrm{Stored}^{\ell}_{I,j}$ stored by node ℓ attains the same value regardless of which node sent it. Thus, node ℓ needs to store only a single copy of $\mathrm{Stored}^{\ell}_{I,j}$, and all stored values are consistent with the execution of NetStore(S, I), independent of the sender.

Retrieving stored information from the network. The NetRetrieve procedure takes as input a key I and aims to retrieve the string S that was previously stored in the network using this key. Toward that goal, the node v that executes NetRetrieve(I) sends I to all other nodes in the network. In return, each node $\ell \in [n]$ replies with a tuple $(j, I, \mathrm{Stored}^{\ell}_{I,j})$ for each j it holds. For each such j, let C_j be the string defined by

$$C_j[\ell] = \begin{cases} c_{I,j} & \text{if } (j, I, c_{I,j}) \text{ was received from } \ell \\ \perp & \text{otherwise} \end{cases} \tag{5}$$

Node v then computes $S_j = \mathrm{Dec}(C_j)$ for all indices j for which C_j is non-empty, and outputs the concatenated string $S = (S_1, S_2, \ldots)$.

Observation 5.3. *Let $S \in [n]^*$ be a string that was successfully stored using key I. Then the procedure NetRetrieve(I) completes in $O(\lceil |S|/n \rceil)$ rounds. Moreover, if the node executing NetRetrieve(I) does not crash during this procedure, it outputs S.*

The above holds because, if S was successfully stored in the network, each non-crashed node holds the corresponding symbol $\mathrm{Enc}(S_j)$ for every j. Consequently, each codeword C_j obtained by the NetRetrieve procedure is missing at most αn symbols due to node crashes. Therefore, the decoder successfully reconstructs the original value, that is, $\mathrm{Dec}(C_j) = S_j$.

Inputs, outputs, states, and bookkeeping. During the simulation algorithm described in the next section, the network stores and retrieves multiple strings corresponding to nodes' states and messages exchanged during the T rounds of the simulated algorithm $\mathcal{A}$. In particular, for each round r of $\mathcal{A}$, we store and retrieve the string $S_j(r)$, which contains all the n messages received by node j in round r of $\mathcal{A}$, and the string $M_j(r)$, which contains the n messages sent by node j in round r of $\mathcal{A}$. The string $S_j(0)$ denotes the input of node j to the algorithm $\mathcal{A}$; as specified in the model, these inputs are already stored in the network when the simulation begins (see Section 2.1).

Since $\mathcal{A}$ is a congested-clique algorithm, each entry of $S_j(r)$ and $M_j(r)$ is a message of size $O(\log n)$ bits. Specifically, $S_j(r)[i]$ denotes the message that node j receives from node i in round r, while $M_j(r)[i]$ denotes the message that node j sends to node i in round r. Without loss of generality, we assume that the output of $\mathcal{A}$ is all messages sent in round T. Equivalently, for each node j, the

string $M_j(T)$ represents its output. Hence, a correct simulation of $\mathcal{A}$ reduces to computing $M_j(T)$ and performing NetStore($M_j(T)$) for all $j \in [n]$.

To simplify the notation and the description of the simulation, we assume a fixed numbering of the $2n(T+1)$ strings $\{S_j(r), M_j(r)\}$. Thus, for each $j \in [n]$ and each $r \in \{0, \ldots, T\}$, the key I corresponding to $S_j(r)$ or $M_j(r)$ is fixed and known to all nodes, and we therefore omit it from the simulation description.

5.2 Simulation Algorithm

Let us overview our task-completion based crash-resilient simulation of a T round (non crash resilient) congested-clique algorithm $\mathcal{A}$. The simulation proceeds in T iterations, where iteration r simulates the r-th round of $\mathcal{A}$. In "simulating" the r-th round, we mean that the simulation stores in the network the state each node $j \in [n]$ holds in the end of the r-th round of $\mathcal{A}$. The *state* of the node j (at the end of round $r \geq 1$) contains its input and all messages it has received until the end of round r.[4] We recall that the j-th input, $S_j(0)$, is already stored in the network at the onset of the simulation. Hence the state of j at the end of round r is the tuple $(S_j(0), \ldots, S_j(r))$.

Somewhat informally, to simulate the r-th round, assuming that the $(r-1)$-st round has been successfully simulated, we perform following steps, carried out in parallel for each $j \in [n]$: (1) retrieving the state of node j, which allows the simulation to compute all messages sent by j in round r of $\mathcal{A}$; (2) computing the messages that j receives in round r of $\mathcal{A}$; and (3) storing the updated state of j back in the network. While this outline is conceptually straightforward, its implementation in the presence of crashes is considerably more complex: node j may have crashed and therefore must be simulated by another node, which itself may also crash, and so on. To address this challenge, we employ the task-completion framework to carry out each of the above steps.

1. To compute all the outgoing messages of node j in round r of $\mathcal{A}$, we define a *task completion instance* in which completing the ℓ-th task consists of the following steps: (1) retrieving the state of node ℓ at the end of round $r-1$, that is, retrieving the strings $S_\ell(0), \ldots, S_\ell(r-1)$; (2) using this state to locally compute $M_\ell(r)$, the set of all messages that node ℓ sends in round r of $\mathcal{A}$; and (3) storing $M_\ell(r)$ in the network (see Task-Completion Instance 1).

2. The next step is to deliver to each node j all messages designated to it in round r, namely the vector $(M_1(r)[j], \ldots, M_n(r)[j])$, assuming that each string $M_i(r)$ has been stored in the network. To this end, we employ two nested task-completion instances.

   The *inner* task (Task-Completion Instance 2) is responsible for retrieving the stored $M_j(r)$ and forwarding its constituent messages $M_j(r)[1], \ldots, M_j(r)[n]$ to the nodes simulating nodes 1 through n, respectively. The identity simulated by each non-crashed node is determined by the *outer* task (Task-Completion Instance 3). In this task, the node simulating identity ℓ constructs $S_\ell(r)$ by collecting the set of messages received by ℓ in round r of $\mathcal{A}$ sent during the inner task.

[4] We remark that one can redefine the state of a node in round i to be the local memory held by the node at that round. While this memory can always be derived from the node's inputs and incoming messages, in some cases it may be significantly shorter. Under this definition, the resulting complexity of our simulation is $O\big(T \cdot \big(\lceil s/(n \log n)\rceil + \log n\big) \log n\big)$, where s denotes the size of the state. In the general case, where the state includes all inputs and incoming messages, we have $s \leq T \cdot n \log n$, which yields the stated complexity.

3. Once all incoming messages of a simulated node ℓ have been collected, the simulating node stores $S_\ell(r)$ in the network. This storage step is performed at the end of the outer task (Task-Completion Instance 3).

The complete simulation is depicted in Algorithm 2.

Algorithm 2 Crash-resilient Simulation Algorithm

Input: A T-round Algorithm $\mathcal{A}$ to simulate. Parameter $\alpha \in (0, 1)$; An error correction code stated in Section 5.1.

1: **for** $r = 1, \ldots, T$ **do**
2: Execute Algorithm 1 on Task-Completion Instance 1 with input r.
3: Execute Algorithm 1 on Task-Completion Instance 3 with input r.
4: **end for**

Task-Completion Instance 1 The Compute-Messages Task

Input: the simulated round r.

To complete task $\ell \in [n]$:
1: **for** $i \in \{0, 1, \ldots, r-1\}$ **do**
2: NetRetrieve($S_\ell(i)$)
3: **end for**
4: As a function of $S_\ell(0), \ldots, S_\ell(r-1)$, compute $M_\ell(r)$: the messages node ℓ sends in round r of $\mathcal{A}$.
5: NetStore($M_\ell(r)$).

▶ The task is complete once $M_\ell(r)$ is successfully stored.

Task-Completion Instance 2 The Inner Task

Input: a vector $(\ell_1, \ldots, \ell_n)$, the simulated round r.

To complete task $\ell \in [n]$**:**
1: NetRetrieve($M_\ell(r)$)
2: **for** each $j \in [n]$ for which $\ell_j \neq \perp$ **do**
3: Send $(\ell, M_\ell(r)[\ell_j])$ to node j. ▷ j simulates ℓ_j.
4: **end for**

▶ The task is complete once every non-crashed node j has received $(\ell, M_\ell(r)[\ell_j])$.

Task-Completion Instance 3 The Outer Task

1: **Input:** the simulated round r.

To complete task $\ell \in [n]$:

2: Broadcast ℓ to all nodes; receive ℓ_j from node $j \in [n]$ or set $\ell_j = \perp$ if no value is received.

3: Execute Algorithm 1 on Task-Completion Instance 2 (the inner task) with input $(\ell_1, \ldots, \ell_n), r$.

4: For any value (j, val_j) received during the execution of the inner task, set $S_\ell(r)[j] \leftarrow \text{val}_j$

5: NetStore($S_\ell(r)$). ▷ The inner task guarantees that all indices of $S_\ell(r)$ are set

► The task is complete once $S_\ell(r)$ is successfully stored.

5.3 Analysis

Towards proving Theorem 5.1 we now analyze the correctness and complexity of Algorithm 2. Our first step is showing that all the task completion instances we define are both idempotent, independent, and executor-safe (see Section 2.2), so we could execute our task-completion Algorithm 1 on each of them. Then, we show by induction that at the end of iteration r of the simulation, $S_\ell(r)$ and $M_\ell(r)$ are stored in the network, for all $\ell \in [n]$. This implies the correct simulation of $\mathcal{A}$, as all messages it sends are correctly computed and communicated, and its output $\{M_j(T)\}_{j\in[n]}$ is stored in the network.

Lemma 5.4. *The tasks defined as part of Task Completion Instances 1–3 are independent, idempotent, and executor-safe.*

Proof. The independence of the inner task (Task-Completion Instance 2) and the compute-message task (Task-Completion Instance 1) easily follows from the fact that these tasks only retrieve information that is already stored in the network, and stores new information not in use by the other tasks. Moreover, no congestion arises even when all n tasks are executed simultaneously, since each node communicates with each of its neighbors at most once per round.

For the independence of the outer task (Task-Completion Instance 3), again the inputs only depend on r and the task itself, and information stored in one task is not an input of any other outer task, nor the any of the inner sub-tasks. Moreover, all tasks can be executed simultaneously without causing congestion. To see this, note that in all lines except for the call to the inner task in line 3, each node communicates with each of its neighbors at most once per round. Line 3 initiates invocation of the task-completion Algorithm 1 on the inner task instance, which is independent as we proved above.

Let us now consider the idempotence of the three task instances. The idempotence of the outer task and the compute-messages task follows from the fact that the NetStore procedure is idempotent conditioned on all the nodes storing the same information. For the inner task instance, note that completing the ℓ-th inner task is defined as the event that all non-crashed nodes j receive the message $(j, M_\ell(r)[\ell_j])$, rather than as the act of a specific node sending these messages. The reason is that different nodes may have different input vectors $(\ell_1, \ldots, \ell_n)$, which may differ only at indices corresponding to crashed nodes (namely: some nodes will see a $\perp$ for a node that crashed while announcing its ℓ in line 2 of Task-Completion Instance 3). Consequently, some nodes may

send the message $(j, M_\ell(r)[\ell_j])$ to a crashed node, while other nodes attempting to complete the ℓ-th inner task may not. Nevertheless, with respect to the non-crashed nodes, the final outcome is identical. Hence, the inner tasks are idempotent.

Finally, we argue that all three task instances are executor-safe. The inner and compute-messages task instances are executor-safe since the node attempting to complete a task executes the NetRetrieve operation by itself (which always succeeds in the presence of up to αn crashes; see Observation 5.3) and subsequently performs the NetStore operation or sends the required messages on its own. The argument for the outer task instance is slightly more involved, as each outer task invokes an inner task completion instance, and thus the completion of an outer task depends on the correct completion of all the corresponding inner task, performed by other nodes. However, by the correctness of Algorithm 1 (Theorem 4.1), we are guaranteed that once line 3 completes, all elements of $S_\ell(r)$ have been delivered to the node v attempting to complete the outer task ℓ, *regardless* of all crashes except for that of v. This directly means that outer tasks are executor-safe. □

By the above lemma, executing Algorithm 1 on each task-completion instance successfully completes all tasks associated with that instance, even in the presence of up to αn crashes. We next show that our crash-resilient simulation (Algorithm 2) correctly simulates any (non-resilient) algorithm $\mathcal{A}$ on a round-by-round basis.

Theorem 5.5. *For any iteration $1 \leq r \leq T$ of Algorithm 2,*

1. *If $S_\ell(0), \ldots, S_\ell(r-1)$ is successfully stored in the network for all $\ell \in [n]$ prior to the execution of Task-Completion Instance 1 (line 2 in Algorithm 2), then at the end of this execution, $M_\ell(r)$ is successfully stored in the network for all $\ell \in [n]$.*

2. *If $S_\ell(0), \ldots, S_\ell(r-1)$ and $M_\ell(r)$ are stored for all $\ell \in [n]$ prior to the execution of Task-Completion Instance 3 (line 3 in Algorithm 2), then at the end of this execution, $S_\ell(r)$ is stored for all $\ell \in [n]$.*

Proof. Item (1) follows from the fact that $M_\ell(r)$ is a function of $S_\ell(0), \ldots, S_\ell(r-1)$. The ℓ-th task in Task-Completion Instance 1 consists of retrieving all strings $S_\ell(0), \ldots, S_\ell(r-1)$, locally computing $M_\ell(r)$, and storing the result in the network. Therefore, once the ℓ-th task is completed, the statement follows. Note that, due to the error-correction code, the retrieval of $S_\ell(0), \ldots, S_\ell(r-1)$ succeeds regardless of which nodes have crashed, provided that the node executing the ℓ-th task does not crash during its execution (Observation 5.3).

For Item (2), assume that some node i executes the ℓ-th task of the outer task and does not crash before completing it. At the beginning of the ℓ-th task, the node i broadcasts the value ℓ to the rest of the network, indicating that it assumes the role of simulating node ℓ in $\mathcal{A}$ for this iteration. Since node i does not crash during the execution of this task, this message is delivered to all non-crashed nodes, and each such node sets $\ell_i = \ell$ when executing line 2 of Task-Completion Instance 3. Recall that in the inner task (Task-Completion Instance 2), the i-th task consists of retrieving $M_i(r)$ (line 1) and sending the pair $(i, M_i(r)[\ell_j])$ to node j. By the correctness of the task completion protocol, all tasks of the inner task are completed by the time the nodes reach line 5 of the outer task (Task-Completion Instance 3). Consequently, at that point, node i is guaranteed to have received the values $(j, M_j(r)[\ell])$ for every $j \in [n]$, which together constitute exactly the state $S_\ell(r)$. Therefore, node i has all the information required to store $S_\ell(r)$ at line 5, and this storage operation succeeds since node i does not crash.

By the correctness of the task completion algorithm (Algorithm 1), all outer tasks are completed by the time the execution reaches line 3 in Algorithm 2, which completes the proof of this case. □

Next, we analyze the round complexity of the simulation.

Theorem 5.6. *Algorithm 2 terminates after $O(T^2 \log n + T \log^2 n)$ rounds.*

Proof. The algorithm executes T iterations, where in each iteration we have one invocation of Algorithm 1 on Task-Completion Instance 1 and one invocation on Task-Completion Instance 3.

Consider a compute-messages task for iteration r. Completing this task requires performing r NetRetrieve operations, each on a string of size $O(n \log n)$ bits (i.e., n symbols), which together take $O(r) = O(T)$ rounds by Observation 5.3. The task then performs a single NetStore operation on a string of size $O(n \log n)$ bits, which takes $O(1)$ rounds by Observation 5.2. Overall, completing a single compute-messages task takes $R_{\text{comp-msg}} = O(T)$ rounds. Therefore, by Theorem 4.1, the round complexity of completing all n compute-messages tasks in the presence of crashes is $O(T \log n)$.

Next, consider the outer task. Broadcasting the identity ℓ requires a single round. Invoking Algorithm 1 on Task-Completion Instance 2 takes $O(\log n)$ rounds by Theorem 4.1; This holds since each of the n inner tasks can be completed in $R_{\text{inner}} = O(1)$ rounds, as each such task consists of a single NetRetrieve operation on a string of size $O(n \log n)$ and a single round of message sending. Finally, storing $S_\ell(r)$ takes $O(1)$ rounds. Overall, completing a single outer task takes $R_{\text{outer}} = O(\log n)$ rounds. Therefore, by Theorem 4.1, the round complexity of completing all n outer tasks using Algorithm 1 is $O(\log^2 n)$.

Summing it all together, each iteration of Algorithm 2 takes $O(T \log n + \log^2 n)$, hence its total round complexity is $O(T^2 \log n + T \log^2 n)$. □

Completing the Proof of Theorem 5.1. The round complexity of Algorithm 2 is established in Theorem 5.6. Correctness follows by induction using Theorem 5.5. Initially, the inputs $S_j(0)$ for all $j \in [n]$ are stored in the network. Assume that, for some round $r \geq 1$, the strings $S_j(0), \ldots, S_j(r-1)$ are stored for all $j \in [n]$. Then, in the next iteration of Algorithm 2, Theorem 5.5(1) guarantees that the messages $M_j(r)$ are successfully computed and stored for all $j \in [n]$, and Theorem 5.5(2) ensures that the corresponding $S_j(r)$ are subsequently stored. After T iterations, all messages sent by $\mathcal{A}$ have been simulated and stored. Recall that the output of $\mathcal{A}$ is $\{M_j(T)\}_{j \in [n]}$ (Section 2.1). It follows from Theorem 5.5(1) that this information is stored in the network at the end of the simulation. Therefore, the execution of $\mathcal{A}$ has been correctly simulated, completing the proof.

Acknowledgments

We thank Keren Censor-Hillel for helpful comments on an initial draft of this manuscript. Orr Fischer is supported in part by the Israel Science Foundation, grant No. 1042/22 and 800/22.

References

[1] Richard J. Anderson and Heather Woll. Algorithms for the certified Write-All problem. *SIAM Journal on Computing*, 26(5):1277–1283, 1997.

[2] John Augustine, Anisur Rahaman Molla, Gopal Pandurangan, and Yadu Vasudev. Byzantine connectivity testing in the congested clique. In *36th International Symposium on Distributed Computing (DISC)*, volume 246, pages 7:1–7:21, 2022.

[3] Philipp Bamberger, Fabian Kuhn, and Yannic Maus. Efficient deterministic distributed coloring with small bandwidth. In *ACM Symposium on Principles of Distributed Computing (PODC)*, pages 243–252, 2020.

[4] Jonathan F. Buss, Paris C. Kanellakis, Prabhakar L. Ragde, and Alex Allister Shvartsman. Parallel algorithms with processor failures and delays. *Journal of Algorithms*, 20(1):45–86, 1996.

[5] Keren Censor-Hillel, Orr Fischer, François Le Gall, Dean Leitersdorf, and Rotem Oshman. Quantum distributed algorithms for detection of cliques. In *13th Innovations in Theoretical Computer Science Conference (ITCS)*, volume 215, pages 35:1–35:25, 2022.

[6] Keren Censor-Hillel, Orr Fischer, Ran Gelles, and Pedro Soto. Two for One, One for All: Deterministic LDC–Based Robust Computation in Congested Clique. In *39th International Symposium on Distributed Computing (DISC 2025)*, volume 356 of *LIPIcs*, pages 20:1–20:19, 2025.

[7] Keren Censor-Hillel, Orr Fischer, Tzlil Gonen, François Le Gall, Dean Leitersdorf, and Rotem Oshman. Fast distributed algorithms for girth, cycles and small subgraphs. In *34th International Symposium on Distributed Computing (DISC)*, volume 179, pages 33:1–33:17, 2020.

[8] Keren Censor-Hillel, François Le Gall, and Dean Leitersdorf. On distributed listing of cliques. In *Symposium on Principles of Distributed Computing (PODC)*, pages 474–482, 2020.

[9] Keren Censor-Hillel and Pedro Soto. Computing in a Faulty Congested Clique. In *29th International Conference on Principles of Distributed Systems (OPODIS)*, volume 361, pages 10:1–10:19, 2025.

[10] Yi-Jun Chang, Manuela Fischer, Mohsen Ghaffari, Jara Uitto, and Yufan Zheng. The complexity of (Δ+1) coloring in congested clique, massively parallel computation, and centralized local computation. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, pages 471–480, 2019.

[11] Bogdan S. Chlebus, Roberto De Prisco, and Alex A. Shvartsman. Performing tasks on restartable message-passing processors. In Marios Mavronicolas and Philippas Tsigas, editors, *Distributed Algorithms*, pages 96–110. Springer Berlin Heidelberg, 1997.

[12] Bogdan S. Chlebus, Leszek Gąsieniec, Dariusz R. Kowalski, and Alexander A. Schwarzmann. Doing-it-All with bounded work and communication. *Information and Computation*, 254:1–40, 2017.

[13] Bogdan S. Chlebus, Leszek Gąsieniec, Dariusz R. Kowalski, and Alex A. Shvartsman. A robust randomized algorithm to perform independent tasks. *Journal of Discrete Algorithms*, 6(4):651–665, 2008. Selected papers from the 1st Algorithms and Complexity in Durham Workshop (ACiD 2005).

[14] Bogdan S. Chlebus and Dariusz R. Kowalski. Randomization helps to perform independent tasks reliably. *Random Structures & Algorithms*, 24(1):11–41, 2004.

[15] David Cifuentes-Núñez, Pedro Montealegre, and Ivan Rapaport. Recognizing hereditary properties in the presence of byzantine nodes. In Andrei Arusoaie, Emanuel Onica, Michael Spear, and Sara Tucci Piergiovanni, editors, *29th International Conference on Principles of Distributed Systems (OPODIS)*, LIPIcs, pages 26:1–26:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.

[16] Sam Coy, Artur Czumaj, Peter Davies, and Gopinath Mishra. Optimal (Degree+1)-coloring in congested clique. In *50th International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 261, pages 46:1–46:20, 2023.

[17] Artur Czumaj, Peter Davies, and Merav Parter. Simple, deterministic, constant-round coloring in congested clique and MPC. *SIAM J. on Computing*, 50(5):1603–1626, 2021.

[18] Seda Davtyan, Roberto De Prisco, Chryssis Georgiou, Theophanis Hadjistasi, and Alexander A. Schwarzmann. Coordinated cooperative task computing using crash-prone processors with unreliable multicast. *Journal of Parallel and Distributed Computing*, 109:272–285, 2017.

[19] Roberto De Prisco, Alain Mayer, and Moti Yung. Time-optimal message-efficient work performance in the presence of faults. In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing*, PODC '94, page 161–172, 1994.

[20] Danny Dolev, Christoph Lenzen, and Shir Peled. "Tri, Tri Again": Finding triangles and small subgraphs in a distributed setting. In *Distributed Computing*, volume 7611, pages 195–209. Springer, 2012.

[21] Cynthia Dwork, Joseph Y. Halpern, and Orli Waarts. Performing work efficiently in the presence of faults. In *Proceedings of the Eleventh Annual ACM Symposium on Principles of Distributed Computing*, PODC '92, page 91–102, 1992.

[22] Antonio Fernández, Chryssis Georgiou, Alexander Russell, and Alex A. Shvartsman. The Do-All problem with Byzantine processor failures. *Theoretical Computer Science*, 333(3):433–454, 2005. Structural Information and Communication Complexity.

[23] Orr Fischer, Tzlil Gonen, Fabian Kuhn, and Rotem Oshman. Possibilities and impossibilities for distributed subgraph detection. In *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 153–162, 2018.

[24] Orr Fischer and Merav Parter. All-to-all communication with mobile edge adversary: Almost linearly more faults, for free. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, PODC '25, page 326–336, 2025.

[25] Chryssis Georgiou, Alexander Russell, and Alex A. Shvartsman. The complexity of synchronous iterative Do-All with crashes. *Distributed Computing*, 17(1):47–63, 2004.

[26] Chryssis Georgiou and Alexander A. Shvartsman. *Do-All Computing in Distributed Systems: Cooperation in the Presence of Adversity*. Springer New York, 2008.

[27] Mohsen Ghaffari and Merav Parter. MST in log-star rounds of congested clique. In *Proceedings of the 2016 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 19–28, 2016.

[28] James W. Hegeman, Gopal Pandurangan, Sriram V. Pemmaraju, Vivek B. Sardeshmukh, and Michele Scquizzato. Toward optimal bounds in the congested clique: Graph connectivity and MST. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, pages 91–100, 2015.

[29] Taisuke Izumi and François Le Gall. Triangle finding and listing in CONGEST networks. In *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, pages 381–389, 2017.

[30] Tomasz Jurdzinski and Krzysztof Nowicki. MST in $O(1)$ rounds of congested clique. In *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 2620–2632, 2018.

[31] P. C. Kanellakis and A. A. Shvartsman. Efficient parallel algorithms can be made robust. In *Proceedings of the Eighth Annual ACM Symposium on Principles of Distributed Computing*, PODC '89, page 211–219, 1989.

[32] Paris Christos Kanellakis and Alex Allister Shvartsman. *Fault-tolerant parallel computation*, volume 401 of *The Springer International Series in Engineering and Computer Science*. Springer Science & Business Media, 1997.

[33] Z. M. Kedem, K. V. Palem, A. Raghunathan, and P. G. Spirakis. Combining tentative and definite executions for very fast dependable parallel computing. In *Proceedings of the Twenty-Third Annual ACM Symposium on Theory of Computing*, STOC '91, page 381–390, 1991.

[34] Z. M. Kedem, K. V. Palem, and P. G. Spirakis. Efficient robust parallel computations. In *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*, STOC '90, page 138–148, 1990.

[35] Janne H. Korhonen. Deterministic MST sparsification in the congested clique. *CoRR*, abs/1605.02022, 2016.

[36] Dariusz Kowalski, Peter M. Musiał, and Alexander A Shvartsman. Explicit combinatorial structures for cooperative distributed algorithms. In *25th IEEE International Conference on Distributed Computing Systems (ICDCS'05)*, pages 49–58. IEEE, 2005.

[37] Manish Kumar. Fault-tolerant graph realizations in the congested clique, revisited. In *Distributed Computing and Intelligent Technology*, volume 13776, pages 84–97. Springer, 2023.

[38] Manish Kumar, Anisur Rahaman Molla, and Sumathi Sivasubramaniam. Fault-tolerant graph realizations in the congested clique. In *Algorithmics of Wireless Networks*, volume 13707, pages 108–122, 2022.

[39] Christoph Lenzen. Optimal deterministic routing and sorting on the congested clique. In *Proceedings of the 2013 ACM Symposium on Principles of Distributed Computing (PODC)*, pages 42–50, 2013.

[40] Zvi Lotker, Boaz Patt-Shamir, Elan Pavlov, and David Peleg. Minimum-weight spanning tree construction in $O(\log \log n)$ communication rounds. *SIAM J. on Computing*, 35(1):120–131, 2005.

[41] Michael Mitzenmacher and Eli Upfal. *Probability and computing: Randomization and probabilistic techniques in algorithms and data analysis*. Cambridge university press, 2017.

[42] Joseph Naor and Ron M. Roth. Constructions of permutation arrays for certain scheduling cost measures. *Random Structures & Algorithms*, 6(1):39–50, 1995.

[43] Krzysztof Nowicki. A deterministic algorithm for the MST problem in constant rounds of congested clique. In *53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 1154–1165, 2021.

[44] Gopal Pandurangan, Peter Robinson, and Michele Scquizzato. On the distributed complexity of large-scale graph computations. In *Proceedings of the 30th on Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pages 405–414, 2018.

[45] Merav Parter. (Delta+1) coloring in the congested clique model. In *45th International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 107, pages 160:1–160:14, 2018.

[46] Merav Parter and Hsin-Hao Su. Randomized (Delta+1)-coloring in O(log* Delta) congested clique rounds. In *32nd International Symposium on Distributed Computing (DISC)*, volume 121, pages 39:1–39:18, 2018.

[47] Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.

[48] Alex A. Shvartsman. Achieving optimal CRCW PRAM fault-tolerance. *Information Processing Letters*, 39(2):59–66, 1991.

Appendix

A Additional Technical Lemmas

Theorem A.1 (Chernoff inequality for independent Bernoulli variables)**.** *Let $X_1, \ldots, X_n$ be mutually independent 0–1 random variables with $\Pr(X_i = 1) = p_i$. Let $X = \sum_{i=1}^{n} X_i$ and set $\mu = \mathbb{E}[X]$. The following holds,*

1. *for $0 < \delta \leq 1$, $\Pr(X \geq (1+\delta)\mu) \leq e^{-\mu\delta^2/2}$*
2. *for $0 < \delta < 1$, $\Pr(X \leq (1-\delta)\mu) \leq e^{-\mu\delta^2/2}$*
3. *for $0 < \delta \leq 1$, $\Pr(|X - \mu| \geq \delta\mu) \leq 2e^{-\mu\delta^2/3}$*
4. *for $R \geq 6\mu$, $\Pr(X \geq R) \leq 2^{-R}$*

For proof, see Theorems 4.4 and 4.5 in [41].

Lemma A.2. *Let $x \in [0, 1)$ and $y \geq 0$. Then $\lfloor x\lceil y \rceil \rfloor \leq \lceil xy \rceil$.*

Proof. Since $y \geq \lceil y \rceil - 1$, we have $xy \geq x(\lceil y \rceil - 1)$. Taking ceiling on both sides, we get $\lceil xy \rceil \geq \lceil x\lceil y \rceil - x \rceil$. Let $k = \lfloor x\lceil y \rceil \rfloor$, and $\epsilon = x\lceil y \rceil - k$. We notice that $\epsilon \in [0, 1)$. We have that

$$\lceil x\lceil y \rceil - x \rceil = \lceil k + \epsilon - x \rceil \geq k = \lfloor x\lceil y \rceil \rfloor,$$

where the first equality follows from the definition of ϵ, the second from the fact that $\epsilon - x > -1$ (since $x \in [0, 1)$), and that k is an integer, and the final equality follows by definition of k. Combining all inequalities together, we get that

$$\lceil xy \rceil \geq \lceil x\lceil y \rceil - x \rceil \geq k = \lfloor x\lceil y \rceil \rfloor,$$

as required. □

B A Lower Bound on the Round Complexity of Task Completion

In this section, we show a very simple $\Omega(\log n / \log\log n)$ lower bound on the round complexity of any task completion algorithm. This bound is essentially the same as the bound proven in [25, 26] and is re-proven here for completeness.

Theorem B.1. *Any deterministic algorithm solving the task completion problem with $M = n$ tasks and $R = 1$ in the presence of αn crash-faults for $\alpha \in (0, 1)$, requires $\Omega(\log n / \log\log n)$ rounds.*

Fix an algorithm $\mathcal{A}$ that solves the task completion problem. We construct an adversarial crash-pattern that guarantees that, at the end of round i of $\mathcal{A}$, there are at least $\lfloor \alpha n / \log^i n \rfloor$ incomplete tasks, for any $i = O(\log n / \log\log n)$. In particular, this implies that $\mathcal{A}$ requires $\Omega(\log n / \log\log n)$ rounds to complete all n tasks. We assume that n is at least as large as some sufficiently large constant $C \leq n$, and for convenience halt when the number of incomplete tasks reduces below C. In particular, we assume that for all relevant values of i,

$$\lfloor \alpha n / \log^i n \rfloor \leq \lfloor \alpha n / \log^{i-1} n \rfloor / 2.$$

Consider the first round. Since there are n incomplete tasks and n nodes, by an averaging argument, there exists a set $T_1 \subseteq [n]$ of size $|T_1| = \lfloor \alpha n / \log n \rfloor$ incomplete tasks for which at most two nodes are assigned for their completion. This follows from the following claim:

Claim B.2. *Assume that in a specific round there are n' non-crashed nodes and $k' \leq n'$ incomplete tasks. Then, there exists a set $T_{n',k'} \subseteq [k']$ of size at least $\lfloor k'/2 \rfloor$ such that each task in $T_{n',k'}$ was attempted to be completed by at most $2n'/k'$ nodes at this round.*

Proof. Assume toward contradiction that at least $\lceil k'/2 \rceil$ tasks were attempted to be completed by at least $2n'/k' + 1$ nodes. Since each non-crashed node attempts at most one task, then we get a contradiction by

$$n' \geq \lceil k'/2 \rceil (2n'/k' + 1) > n'.$$

□

In the first round of $\mathcal{A}$, the adversary crashes all nodes that attempt to complete tasks in a set $T_1 \subseteq T_{n,n}$, whose existence follows from the above claim. Since $|T_1| = \lfloor \alpha n / \log n \rfloor$, then $2|T_1|$ bounds the number of nodes crashed in this round. As a result of this crash pattern, all tasks in T_1 remain incomplete at the end of the round. Indeed, for each task in T_1, at most two nodes attempt to complete it, and these nodes are crashed by the adversary.

Next, in the second round of $\mathcal{A}$, Claim B.2 guarantees that there exists a set $T_2 \subseteq T_{n-2|T_1|,|T_1|} \subseteq T_1$ of size $|T_2| = \lfloor \alpha n / \log^2 n \rfloor \leq |T_1|/2$, for which at most $2n/|T_1| \leq \frac{4 \log n}{\alpha}$ nodes are assigned for their completion. The adversary crashes the set of nodes attempting to complete a task in T_2, resulting in at most $(2n/|T_1|) \cdot |T_2| \leq 4n/\log n$ crashes, and all tasks in the set T_2 being incomplete at the end of the second round.

More generally for $i > 1$, in round i, we assume that there is a set T_{i-1} of size $|T_{i-1}| = \lfloor \alpha n / \log^{i-1} n \rfloor$ that is incomplete. By Claim B.2, there exists a set $T_i \subseteq T_{i-1}$ of size $|T_i| = \lfloor \alpha n / \log^i n \rfloor \leq |T_{i-1}|/2$ tasks for which at most $2n/|T_{i-1}| \leq \frac{4 \log^{i-1} n}{\alpha}$ nodes are assigned for their completion. The adversary crashes the set of nodes attempting to complete a task in T_i, resulting in at most $(2n/|T_{i-1}|) \cdot |T_i| \leq 4n/\log n$ crashes, and all tasks in the set T_i being incomplete at the end of round i.

We get that $|T_i| < C$ when $i = \Omega(\log n / \log \log n)$. Denote by r the first round where $|T_r| < C$; then $r = \Theta(\log n / \log \log n)$. Moreover, denoting $T_0 = [n]$, we get that the number of crashes in total is at most

$$\sum_{i=1}^{r} (2n/|T_{i-1}|) \cdot |T_i| \leq \sum_{i=1}^{r} \frac{4 \log^{i-1} n}{\alpha} \left\lfloor \alpha n / \log^i n \right\rfloor \leq \sum_{i=1}^{r} 4n/\log n \leq \alpha n.$$